Algorithms - Week 2 Quiz

Version A

Name: _____

5 points total

Each question is worth 1 point. Show enough work that your answer can be interpreted.

1. Trace the following function.

The print statement is instrumentation: complete the trace table with exactly the values printed on each iteration, then give the returned value.

```
def count_increases(xs):
    count = 0
    for i in range(len(xs) - 1):
        if xs[i + 1] > xs[i]:
            count += 1
        print(i, xs[i], xs[i + 1], count)
    return count

xs = [8, 11, 15, 12, 12, 19, 7]
```

i	xs[i]	xs[i + 1]	count

Returned value: _____

2. Trace this binary search for the target shown below.

Complete one row for every time the loop body runs. This search is unsuccessful; give the final returned value as well.

```
def binary_search(needle, haystack):
    lo = 0
    hi = len(haystack)
    while lo < hi:
        mid = (lo + hi) // 2
        print(lo, hi, mid, haystack[mid])
        if haystack[mid] == needle:
            return mid
        elif haystack[mid] < needle:
            lo = mid + 1
        else:
            hi = mid
    return None

needle = 20
```

idx	0	1	2	3	4	5	6	7	8	9	10	11	12
val	3	7	12	18	23	29	34	41	47	53	58	64	71
idx	13	14	15	16	17	18	19	20	21	22	23	24	25
val	76	82	89	95	101	108	114	121	127	134	141	149	156

lo	hi	mid	haystack[mid]

Returned value: _____

Algorithms - Week 2 Quiz

Version A

Name: _____

5 points total

Each question is worth 1 point. Show enough work that your answer can be interpreted.

3. Recall the timer from the flipped reading.

In the reading I mentioned a better alternative to time.time() for accurately measuring elapsed time (with better accuracy on some architectures). What function did I say you should memorize?

Answer: time._____()

4. Diagnose this timing code.

For this question only, pretend time.time() is our timer; I am using it so I do not give away Question 3. The goal is to measure only how long binary_search takes. The following code produces misleading times. What mistake from the reading was made, and how should it be fixed?

```
before = time.time()
data = make_random_list(100000)
data.sort()
result = binary_search(73129, data)
after = time.time()
elapsed = after - before
```

Mistake and fix:

Quiz 3

Name: _____ / _____

1.

A warehouse has shelves numbered 100 through 199. Each shelf has positions numbered 1 through 20. Write Python code that displays every shelf/position combination. Any order is fine.

2.

This code is supposed to time `do_experiment()` separately 1000 times.

```
import time

times = []

before = time.perf_counter()

for trial in range(1000):
    do_experiment()    # Assume this line works and is correct.
    after = time.perf_counter()
    times.append(after - before)
```

a) What does this code do incorrectly? What do the values added to `times` actually represent?

b) Fix the code.

3. Logarithms

Rewrite the following exponential equation using logarithm notation. Do not evaluate it, and do not use a calculator.

$$5\text{^}x = 47256$$

---

4. Logarithm word problem

A tournament starts with 73,842 players. After each round, about one quarter of the players remain. Write a logarithm that gives approximately how many rounds it will take to get down to one player, the nearest integer is fine. Do not evaluate it, and do not use a calculator.

---

5. Math words

For each expression, write the **most specific** word from the list that describes it. One word will not be used.

constant	logarithmic	linear	quadratic	polynomial	exponential

a.	y = 17	_____
b.	y = 5 log(n)	_____
c.	y = 4n + 3	_____
d.	y = n^5	_____
e.	y = 3^n	_____

6. Polynomial vs. exponential

How do you tell the difference between a polynomial expression and an exponential expression?

Week 05 Quiz Name: _____

1. Consider this code. What is printed?

```
words = ["cat", "dog", "owl", "bat", "fox", "ant",
"cow"]


a = words
b = words


a[1] = "BLUE"
b[4] = a[2]


print(words[1])
print(words[2])
print(words[4])
```

2. Draw the memory diagram after this code has finished running.

```
nums = [10, 20, 30, 40]


a = nums
b = [10, 20, 30, 40]


a[2] = 99
b[0] = nums[1]
```

3. Consider this code. What is printed?

```
class Pet:
    def __init__(self, name):
        self.name = name
        self.friend = None


cat = Pet("Miso")
dog = Pet("Rex")


cat.friend = dog
dog.name = "Buddy"


print(cat.name)
print(dog.name)
print(cat.friend.name)
```

4. Draw the memory diagram after this code has finished running.

```
class Box:
    def __init__(self, label):
        self.label = label
        self.next = None


first = Box("red")
second = Box("green")
third = Box("blue")


first.next = second
second.next = third
third.label = "gold"
```

5. Label line costs on/near the code, combine loop and serial costs, and circle the final Big-O cost.

```
def mystery(data):
    total = 0

    for x in data:
        total = total + x

    for x in data:
        for y in data:
            total = total + x + y

    return total
```

6. Label line costs on/near the code, combine loop and serial costs, and circle the final Big-O cost.

Python list costs: append/pop/data[i]/len = O(1); insert(0, x)/pop(0) = O(n).

```
def mystery(data):
    result = []

    for x in data:
        result.append(x)

    for x in data:
        result.insert(0, x)

    for x in data:
        result.append(x)

    return result
```

Algorithms Quiz 6

Name: _____

1.

The following code creates a linked list of five nodes:

```
head = LLN("P")
head.next = LLN("C")
head.next.next = LLN("H")
head.next.next.next = LLN("V")
head.next.next.next.next = LLN("V")
```

Then this code runs:

```
p = head.next
q = p.next

p.next = q.next
q.next = head
head = q

while head is not None:
    print(head.val)
    head = head.next
```

What does the code print?

2.

Suppose s is initially an empty stack. What does the code print?

```
s.push("A")
s.push("B")
print(s.pop())

s.push("C")
s.push("D")
s.push("E")
print(s.pop())
print(s.pop())

s.push("F")
print(s.pop())
print(s.pop())
```

3.

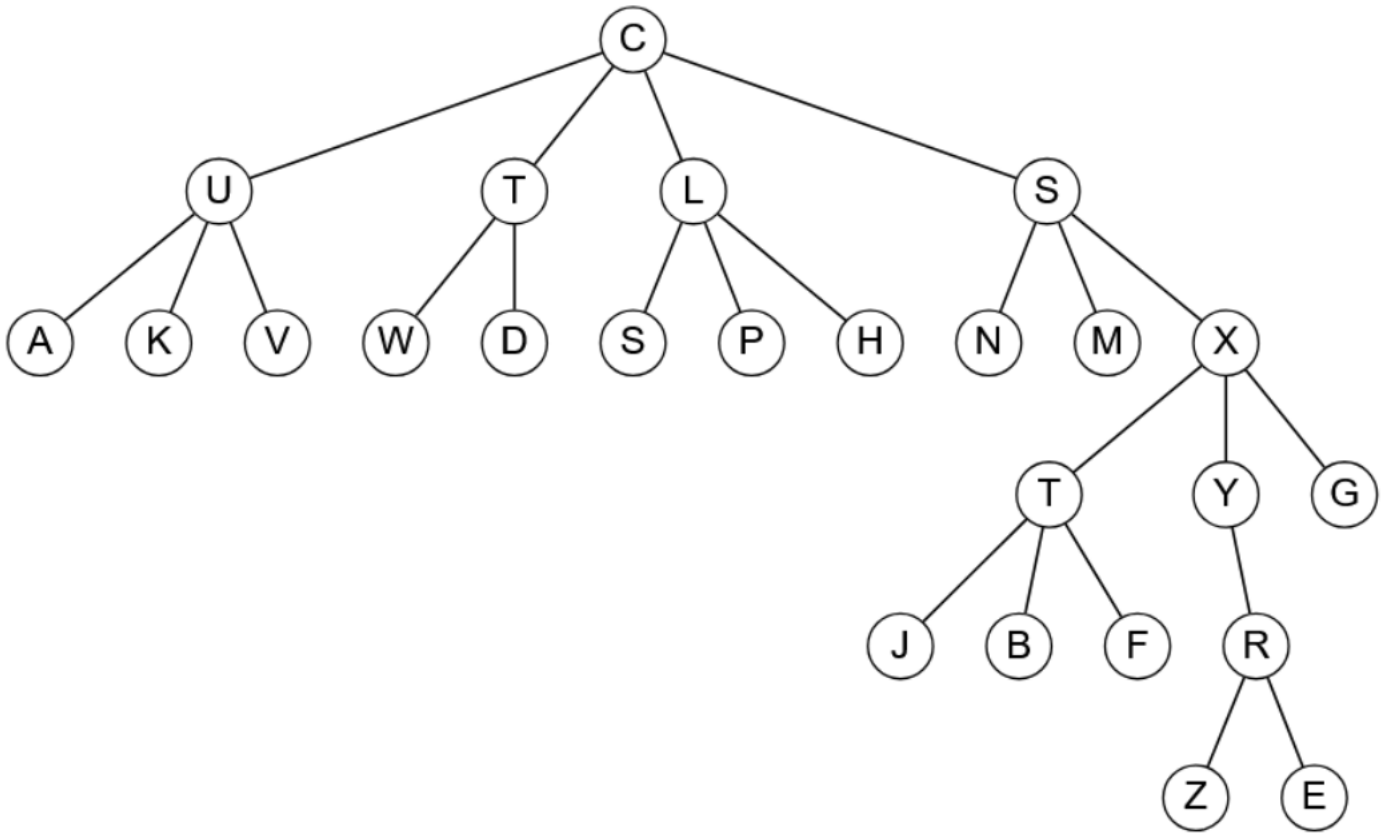
Suppose q is initially an empty queue. What does the code print?

```
q.insert("G")
q.insert("H")
print(q.remove())

q.insert("J")
q.insert("K")
q.insert("L")
print(q.remove())
print(q.remove())

q.insert("M")
print(q.remove())
print(q.remove())
```

Questions 4–7 use the tree below.



4.
How many children does node X have?

5.
What are the depth and height of node X?

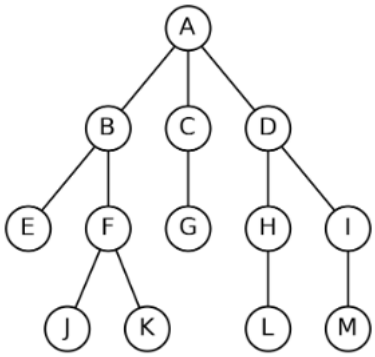
6.
Write the forward pre-order DFS of the tree.

7.
Write the BFS traversal of the tree.

Week 7 Quiz

Name: _____

Questions 1 and 2 use the tree below. Each node has an ordered list of children, from left to right.



1.

The following function receives the root of a tree.

```
def mystery(root):
    pending = [root]
    answer = 0

    while pending:
        node = pending.pop()

        if len(node.children) >= 2:
            answer += 1

        pending.extend(node.children)

    return answer
```

a. What does mystery(root) return for the tree above?

b. Describe, in plain English, what this function calculates for any tree.

2.

What list is returned by mystery(root) for the tree above? Assume root refers to node A. Write your answer as a Python list, e.g., [3, 7, 2].

```
def mystery(root):
    pending = [(root, 0)]
    awesome = []

    while pending:
        node, flavour = pending.pop()

        if flavour == 2:
            awesome.append(len(node.children))

        for child in node.children:
            pending.append((child, flavour + 1))

    return awesome
```

Week 7 Quiz

3.

Consider this code:

```
start = 3

def outer(x):
    total = x + 8
    saved = inner(total + 2)
    return saved + x

def inner(total):
    x = total * 2
    saved = x - 5
    # STOP HERE
    return saved

answer = outer(start)
```

Draw ONE memory diagram showing the state of memory at the exact moment execution reaches the # STOP HERE comment.

Include the global frame and every active function frame. Show all variables in each frame at that moment, and their contents.

Do not draw a sequence of diagrams or the state after the program finishes. Draw only one snapshot at the marked moment.

4.

Consider this code. What is printed?

```
def mystery(n):
    if n <= 0:
        return "!"

    if n % 2 == 0:
        return mystery(n - 1)

    tail = mystery(n - 2)
    return str(n) + tail

print(mystery(8))
```

Reminder: str(n) converts a number into a string. Adding two strings concatenates them.